Assembly Guide - Sukabumi Site

Indonesia ORC Deployment — Spring 2026

American Red Cross / Palang Merah Indonesia

Version 3.4 | Audience: Technical installers | Built 2026-04-09

Contents

Assembly Guide - Sukabumi Site **2**
Table of Contents . . . 2
Pre-Assembly Checklist . . . 3
Component Inventory . . . 4
Assembly Steps . . . 6
Power-On Procedure . . . 27
ORC Software Configuration . . . 28
Troubleshooting . . . 34
Maintenance Notes . . . 35
Site-Specific Configuration . . . 35
Station Commissioning: Camera Survey & Calibration . . . 36

Assembly Guide - Sukabumi Site

Site: Sukabumi, Indonesia (foothills) **Type:** Solar-powered, PoE camera with built-in IR (power-cycled with Pi) **Purpose:** Replacement of failed river monitoring unit

---

Table of Contents

- Pre-Assembly Checklist
  - 1. Software Configuration
  - 2. Hardware Testing (Dry-Fit)
  - 3. Conformal Coating
- Component Inventory
- Assembly Steps
  - Step 1: Prepare Mounting Plate and DIN Rails
  - Step 2: Assemble Compute Stack
  - Step 3: Mount Components on DIN Rail
  - Step 4: Wire Power Distribution
  - Step 5: Wire PoE Camera Circuit
  - Step 6: Connect Peripherals
  - Step 7: Test Mounting Plate Assembly
  - Step 8: Prepare Enclosure and Install Bulkheads
  - Step 9: Install Mounting Plate and Connect External Peripherals
  - Step 10: Configure Pi Camera Network
  - Step 10b: Prepare USB Storage
  - Step 10c: Configure NTP Server for Camera Time Sync
  - ~~Step 10d: FTP Server~~ — removed, not used (see ISS-003)
  - Step 11: Configure PoE Camera Settings
  - Step 11b: Enable Capture Service
  - Step 11c: Enable Sensor Logging Service
  - Step 12: Mount External Components
  - Step 13: Final Assembly and Sealing
- Power-On Procedure
- ORC Software Configuration
  - ORC-OS Web UI Initial Setup
  - Capture Scheduling
  - Witty Pi 5 Power Management
  - Pangolin Remote Access
  - LiveORC Server Check
  - End-to-End Verification
  - In-Country TODOs
  - Sensor Field Testing
- Troubleshooting
- Maintenance Notes
- Site-Specific Configuration
- Station Commissioning: Camera Survey & Calibration

Pre-Assembly Checklist

Complete these steps BEFORE traveling to Indonesia:

1. Software Configuration

- ☐ Flash ORC-OS image to MicroSD card using Raspberry Pi Imager
  - Follow the ORC-OS README — section **"Getting the image on the SD card"**
  - Use the `.img.gz` file provided by Rainbow Sensing (do NOT unpack it)
  - In Pi Imager: choose Pi 5 as device, "Use custom" for OS, select the `.img.gz` file
  - When prompted for OS customisation: set hostname (e.g. `orc-sukabumi`), keep username as `pi`, enable SSH, skip WiFi (we use Ethernet/LTE)
  - **WARNING:** Do NOT change the username from `pi` — this will break ORC-OS
- ☐ Boot Pi 5 and verify ORC-OS runs
  - Follow the ORC-OS README — section **"Test your installation"**
  - First boot takes 2-3 minutes (services compile, filesystem expands, then auto-reboots)
  - After reboot, navigate to `http://<hostname>.local` to verify the ORC-OS web dashboard loads
  - Set the ORC-OS web dashboard password when prompted
- ☐ Load Witty Pi 5 schedule files via laptop USB:
  1. Shut down the Pi (Witty Pi stays powered from external supply)
  2. Connect laptop USB cable to Witty Pi's USB-C port
  3. Copy `.wpi` files from `spring_2026_ID/pi/sukabumi/` into `schedule/` on the emulated drive
  4. Safely eject the drive from the laptop, disconnect cable
  5. Boot the Pi, run `wp5` option 6 to select the active schedule
  - **Do NOT connect the Pi's USB-A to the Witty Pi USB-C** — causes reboot loop
- ☐ Configure Witty Pi 5 HAT+ wake schedule via `wp5` interactive menu (15-minute cycle)
  - Witty Pi owns wake/startup; ORC-OS owns shutdown (`shutdown_after_task`)
  - Verify `orc-api.service` has `After=... wp5d.service` (set by `deploy.sh`)
- ☐ Configure Pi eth0 static IP (192.168.50.1/24) and dnsmasq DHCP for camera network
- ☐ **Do NOT change timezone from UTC** — ORC-OS requires UTC (see REBOOT_CHECKLIST.md)
- ☐ Install and configure chrony as NTP server for camera network
- ☐ Format USB drive as ext4, mount at /mnt/usb, add to fstab
- ~~Configure vsftpd~~ — not needed; capture is via RTSP pull (orc-capture)
- ☐ Configure camera via camtool.py (1080p, 16Mbps CBR, IR-only, NTP from Pi)
- ☐ Insert and format MicroSD in camera, set recording to continuous (CMR)
- ☐ Test video capture pipeline (download 5s clip from camera SD via RTSP)
- ☐ Enable sensor logging (`orc-sensors.timer`) — see Step 11c
- ☐ Verify SHT40 readings in `/var/log/orc/sensors/sht40_*.csv`
- ☐ ~~Configure WiFi hotspot~~ N/A (hotspot not available; use Pangolin or Tailscale for remote access)
- ☐ Set up WS2812B NeoPixel status LED (GPIO 18 data, 5V/GND from Pi rail)
- ☐ Pre-configure Telkomsel APN (if known)

2. Hardware Testing (Dry-Fit)

Assemble, wire, and test the complete system **before** conformal coating. You need to verify all connections work and identify every contact point that must be masked during coating.

- ☐ Test Pi 5 + Witty Pi 5 HAT+ + Geekworm G469 stack boots correctly (3-board stack)
- ☐ Verify USB flash drive is recognized and mounted at /mnt/usb
- ☐ Test LTE modem connects (with test SIM)
- ☐ Test video capture pipeline (5s clip from camera SD → /mnt/usb/incoming)
- ☐ Test PoE switch powers camera when relay energized
- ☐ Test SHT40 sensor: `i2cdetect -y 1` shows 0x44, `orc-sensors` returns readings
- ☐ Verify WS2812B NeoPixel cycles R/G/B via GPIO 18

3. Conformal Coating (After Testing, Before Travel)

Apply conformal coating only after the full system has been assembled, tested, and verified working. This ensures you know exactly which contact points need to be masked — coating over a connection you missed will cause failures that are difficult to diagnose in the field.

Do this in a low-humidity environment. Allow 24 hours cure time.

Boards to Coat: - [ ] Raspberry Pi 5 - [ ] Witty Pi 5 HAT+ PCB - [ ] Geekworm G469 terminal block HAT

Masking (Use Kapton tape on all contact points): - GPIO header pins (all 40 pins — Pi 5, Witty Pi 5, and G469) - USB-A and USB-C ports - HDMI ports - Ethernet port - MicroSD card slot - Witty Pi 5 CR2032 battery holder - J2 power button pigtail Dupont connectors (mask connectors; coat solder joints) - Heat sink mounting holes / thermal pad contact area - Any other connector or contact point used during testing

Procedure: 1. Disassemble the tested stack (remove G469 from Pi) 2. Clean boards with IPA wipe, let dry completely 3. Apply Kapton tape to all masked areas listed above 4. Brush thin, even coat of MG 422C silicone on exposed PCB areas 5. Allow 24 hours cure time at room temperature 6. Remove Kapton tape after cure 7. Inspect for complete coverage, recoat bare spots if needed 8. Label each board "COATED" with date 9. Reassemble the stack and verify it still boots

---

Component Inventory

Verify all components before starting assembly:

Compute Stack

- ☐ Raspberry Pi 5 8GB (coated)
- ☐ Witty Pi 5 HAT+ (coated) — RTC with CR2032 coin cell, I2C (0x51), passes through all 40 GPIO pins
- ☐ Geekworm G469 HAT (coated)
- ☐ SanDisk 256GB USB flash drive
- ☐ MicroSD card 64GB (with OS)
- ☐ Heatsink/cooler for Pi 5
- ☐ CR2032 coin cell for Witty Pi 5 HAT+ RTC
- ☐ 16mm standoffs (for 3-board stack: Pi 5 bottom, Witty Pi 5 middle, G469 top)

Connectivity

- ☐ Quectel EG25-G modem + EXVIST Mini PCIe-USB adapter

- ☐ Proxicast ANT-122-S02 MIMO LTE puck antenna (IP67, 12mm hole mount)

PoE Camera System

- ☐ ANNKE C1200 PoE camera (12MP, built-in IR, factory-sealed IP67)
- ☐ LINOVISION Industrial PoE Switch (Gigabit, 12V DC input)
- ☐ Electronics-Salon 4-channel SPDT DIN Rail relay module (GPIO-triggered via G469)
- ☐ DDR-60G-5 DC-DC converter (12V→5V for Pi 5 via hardwired 5V/GND GPIO)
- ☐ DDR-60G-12 DC-DC converter (12V→12V regulated for PoE switch)
- ☐ Cat6 outdoor shielded cable (to camera)
- ☐ CNLINKO weatherproof ethernet bulkhead, IP67 (enclosure feedthrough)
- ☐ Pole mount bracket (stainless steel)

Climate Monitoring

- ☐ SHT40 temperature/humidity sensor (I2C, inside enclosure)

User Interface

- ☐ WS2812B (NeoPixel) individual RGB pixel PCB (1 per site)
- ☐ Clear cast acrylic sheet (small offcuts, ~2-3mm thick)
- ☐ Clear neutral-cure silicone sealant (GE Silicone II or equivalent — NOT acetoxy)
- ☐ 1× momentary pushbutton, normally open (power). Any panel-mount momentary NO switch works — see GPIO_WIRING.md Step 5 for requirements and options. Match hole size to the button you purchase.

Enclosure & Mounting

- ☐ Outdoor enclosure (~300×200×150mm)
- ☐ DIN rail 35mm (×1-2)
- ☐ DIN rail clips for Pi
- ☐ Terminal blocks
- ☐ SP13 weatherproof DC power bulkhead (IP68, for 12V input)
- ☐ CNLINKO weatherproof ethernet bulkhead (IP67, for PoE camera)
- ☐ Gore M12 vents (×2 for enclosure)

Rain Gauge

- ☐ Hydreon RG-15 optical rain gauge (UART RS232 TTL 3.3V)
- ☐ Built-in mounting holes (no separate bracket needed)
- ☐ Cable (5m)

Hardware & Consumables

- ☐ Stainless steel hardware kit
- ☐ Cable ties (UV-resistant)
- ☐ Dielectric grease
- ☐ Silicone sealant

Assembly Steps

Step 1: Prepare Mounting Plate and DIN Rails

Complete all mounting plate work before drilling the enclosure box. The mounting plate is where all DIN rail components live. Get everything mounted, wired, and tested on the plate first — then drill the enclosure for bulkheads, LEDs, and buttons once you know the layout works.

Tools needed: Drill, marker, saw (for cutting DIN rail), screwdriver

1. **Mark and drill DIN rail mounting holes on the plate:**
   - Remove the mounting plate from the enclosure
   - Mark positions for two horizontal DIN rails
   - Drill screw holes for rail mounting
2. **Cut and install DIN rails:**
   - Cut rails to fit plate width
   - Mount horizontally using screws
   - Leave clearance for cable routing below

Step 2: Assemble Compute Stack (15 min)

Tools needed: Phillips screwdriver

1. **Stack order (bottom to top):**

   ```
   [Raspberry Pi 5]         (bottom)
        ↑
   [Witty Pi 5 HAT+]       (middle — RTC, CR2032, I2C 0x51)
        ↑
   [Geekworm G469 HAT]     (top — screw terminals)
   ```

   3-board stack with 16mm standoffs. The Witty Pi 5 HAT+ passes through all 40 GPIO pins, so the G469 terminal block HAT works unchanged on top.

   Why Witty Pi 5 instead of Pi 5 built-in RTC: The Pi 5 ML-2020 battery connector (J5) broke on both Sukabumi and Jakarta boards — the Molex connector cannot handle any mechanical stress. The Witty Pi 5 uses a standard CR2032 coin cell holder, which is far more robust.

2. **Assembly:**
   - Install heatsink on Pi 5 CPU (thermal pad contact)
   - Install CR2032 coin cell in Witty Pi 5 HAT+ battery holder
   - Seat Witty Pi 5 HAT+ onto Pi 5 GPIO header, secure with 16mm standoffs
   - Seat Geekworm G469 onto Witty Pi 5 header
   - Press down firmly until fully seated
   - Secure stack with standoffs
3. **Verify:**
   - All headers fully seated
   - No bent pins
   - Heatsink secure

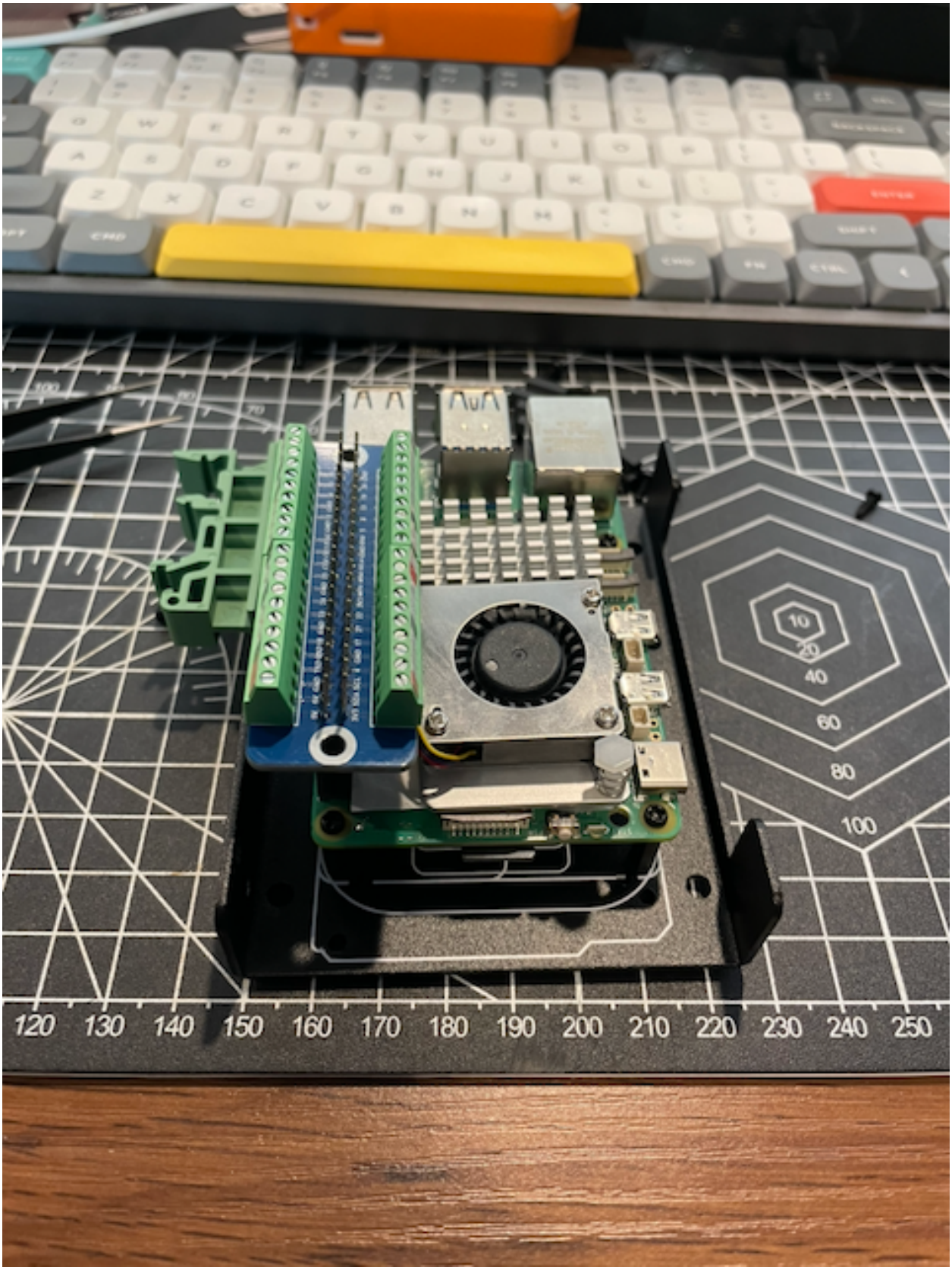

Figure 1: Completed Pi 5 compute stack, side view showing GPIO terminal riser, active cooler, and DIN rail bracket

Figure: Completed compute stack ready for DIN rail mounting. (Photo taken before Witty Pi 5 HAT+ was added. Your assembly will have three boards: Pi 5 + Witty Pi 5 + G469.)

Step 3: Mount Components on DIN Rail (20 min)

Tools needed: Screwdriver, DIN rail clips

1. **Mount Pi stack:**
   - Attach DIN rail clip to Pi mounting holes
   - Snap onto DIN rail
   - Verify secure fit
2. **Mount DIN rail components:**
   - LINOVISION PoE switch
   - Electronics-Salon relay module
   - DDR-60G-5 buck converter (12V→5V)
   - DDR-60G-12 buck converter (12V→12V regulated)
   - Terminal blocks (snap onto rail)
   - Fuse holder (snap onto rail or screw mount)

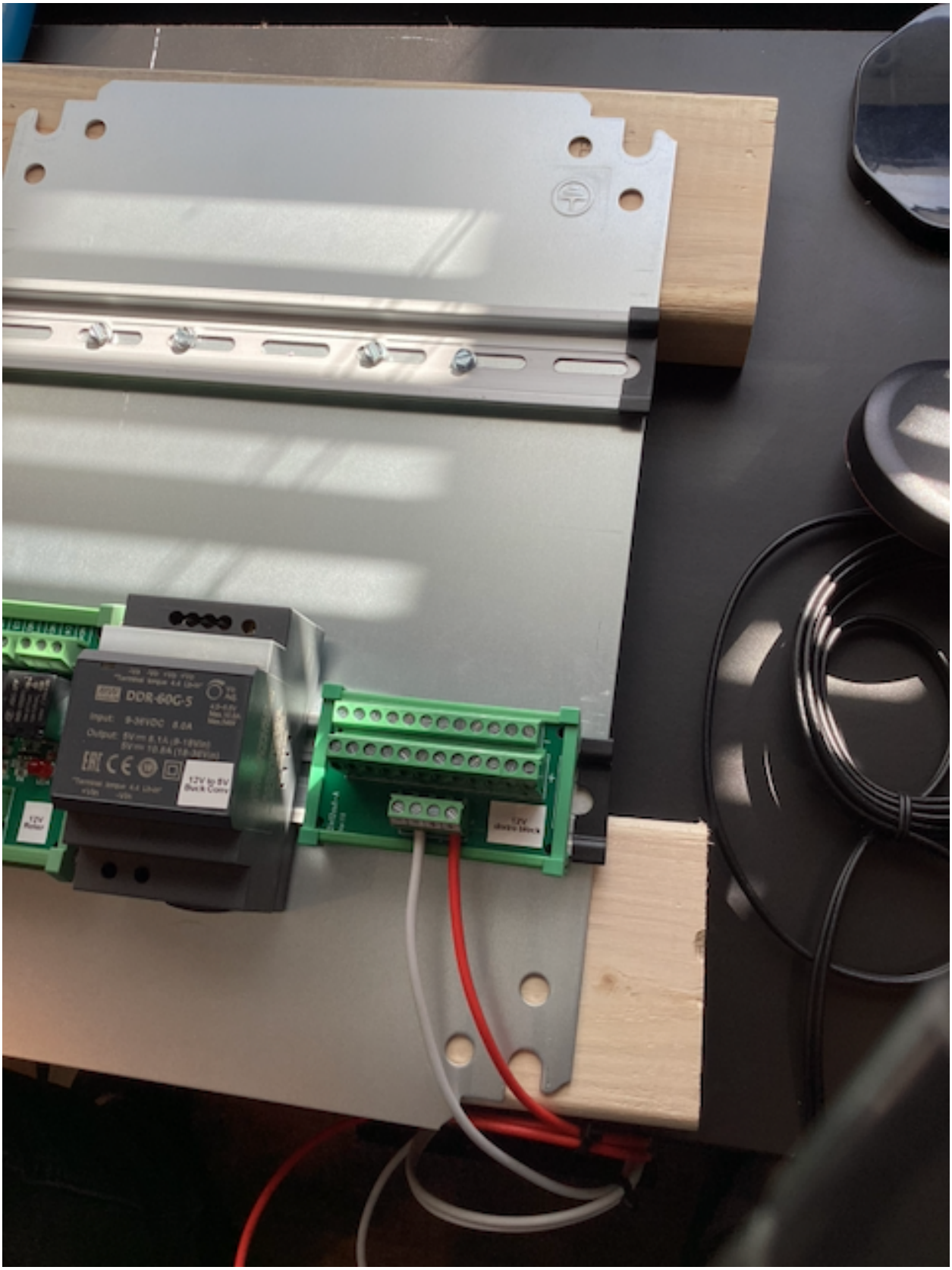

Figure 2: First DIN rail mounted with relay module and DDR-60G-5 buck converter

Figure: First components snapped onto DIN rail.

3. **Mount items without DIN rail clips:**
   - Quectel modem: sits inside the EXVIST WWAN USB carrier, which is Velcro/Dual-Lock attached to a DIN rail clip
   - Small breakout boards (SHT40, etc.): double-sided tape to a nearby DIN-mounted component's carrier tray or flat surface
4. **Layout:**

```
┌──────────────────────────────────────┐
│  [Gore Vent]              [Gore Vent]  │
│              [Puck Antenna]            │
│                                        │
│  ┌──────────────────────────────┐      │
│  │         DIN RAIL             │      │
│  │ [Pi Stack] [PoE Sw] [Relay]  │      │
│  │ [DDR-5] [DDR-12] [Fuse] [Term]  │   │
│  └──────────────────────────────┘      │
│                                        │
│  [Modem]                               │
│  (velcro)                              │
│                                        │
│  ○ ○ ○  [●] [●]                         │
│  LEDs   Mnt  Pwr                       │
└──────────────────────────────────────┘
```

Step 4: Wire Power Distribution (20 min)

Tools needed: Wire strippers, screwdriver

See **GPIO_WIRING.md** for detailed step-by-step wiring instructions, pin assignments, continuity verification checklists, and build photos.

1. **Power distribution:**

```
Solar 12V ──┬── Inline Fuse (5A) ── DDR-60G-5 (12V→5V) ──► hardwired 5V/GND ──► Pi 5 GPIO
            │
            └── Inline Fuse (5A) ── DDR-60G-12 (12V→12V reg) ──► Relay CH1 ──► PoE Switch

Note: DDR-60G converters regulate voltage from battery
(which varies 10-14V depending on charge state).
DDR-60G-5 provides clean 5V hardwired to Pi 5 GPIO pins.
Witty Pi 5 HAT+ provides RTC (CR2032 coin cell) for wake scheduling.
PoE switch receives regulated 12V through relay channel 1 (GPIO 24).
Camera boots when Pi wakes and closes relay, powers down when relay opens.
```

2. **Terminal block connections:**
   - Use solid core wire (18-22 AWG) for all internal DIN rail wiring
   - Label all terminals (12V+, 12V-, GND, etc.)

Step 5: Wire PoE Camera Circuit on Mounting Plate (15 min)

Tools needed: Screwdriver, Ethernet cable

1. **Connections (all on the mounting plate):**
   - 12V+ from terminal block → fuse holder input
   - Fuse holder output → relay CH1 COM input
   - Relay CH1 NO output → PoE switch 12V+ input
   - PoE switch GND → terminal block GND
   - Relay module VCC → G469 Pin 4 (5V)
   - Relay module GND → G469 Pin 20 (GND)
   - Relay IN1 → G469 GPIO 24
   - Short Ethernet patch cable: PoE switch uplink port → Pi 5 Ethernet
2. **Operation (active-high relay logic — verified empirically 2026-03-26):**
   - Pi wakes (via RTC schedule), drives GPIO 24 HIGH → relay CH1 energizes → NO closes
   - 12V flows to PoE switch
   - PoE switch provides 48V PoE to camera over Ethernet
   - Camera boots (~45-60s), built-in IR activates automatically at night
   - Pi captures 5s video via RTSP pull (orc-capture script)
   - Pi drives GPIO 24 LOW → relay de-energizes → NO opens → camera powers down
   - Pi enters sleep via RTC until next scheduled wake

Step 6: Connect Peripherals on Mounting Plate (15 min)

1. **USB flash drive:**
   - SanDisk 256GB directly into Pi 5 USB-A 3.0 port (blue)
2. **LTE Modem:**
   - USB cable from EXVIST adapter → Pi 5 USB 2.0 port
   - SMA pigtails from modem (antenna connects later during enclosure step)
3. **SHT40 sensor (inside enclosure):**
   - STEMMA QT to bare wire cable to Geekworm G469:
     - VCC → 3.3V
     - GND → GND
     - SDA → GPIO 2
     - SCL → GPIO 3

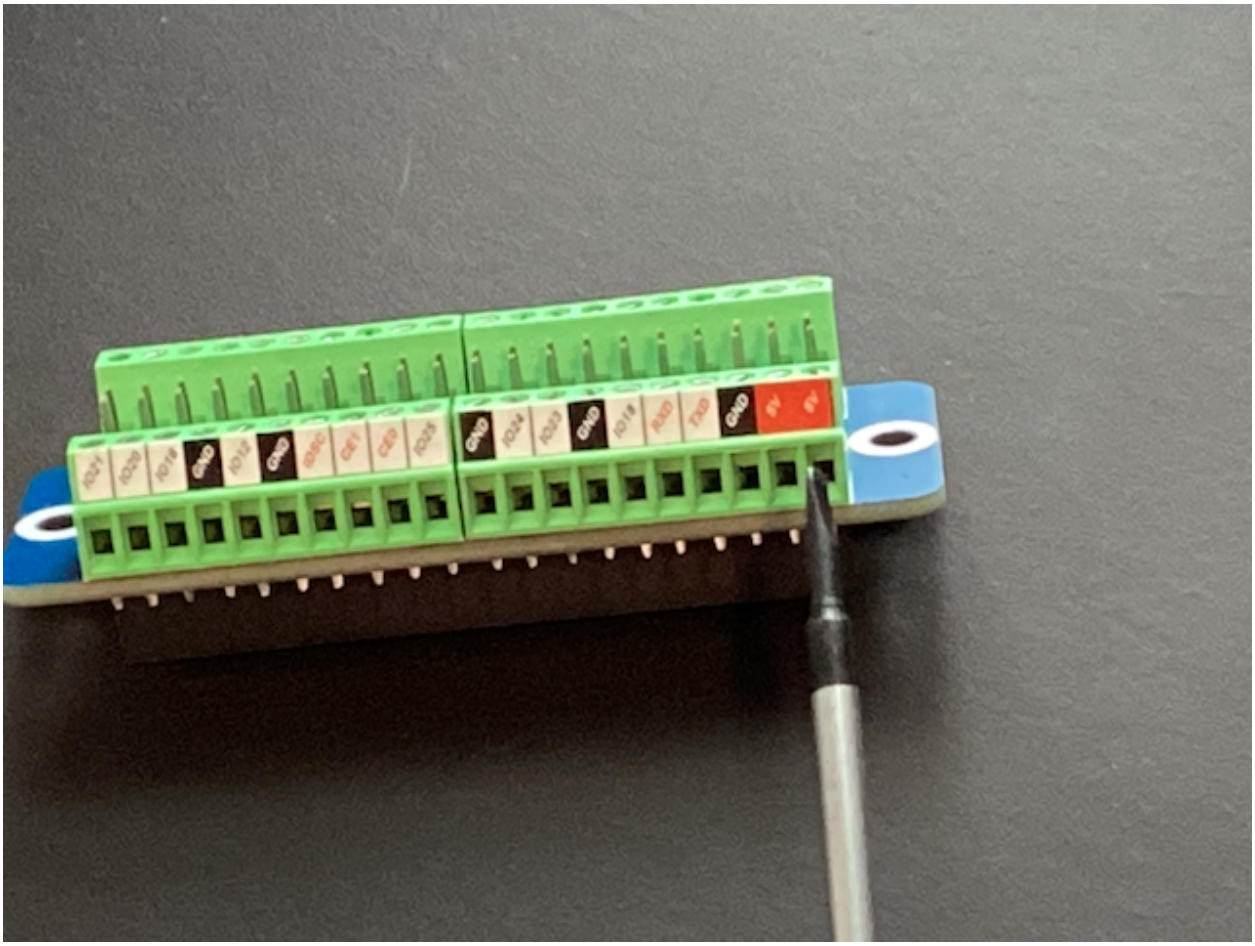

Figure 3: G469 GPIO breakout board showing labeled screw terminal pin assignments

Figure: G469 pin labels for reference during wiring. (Photo taken before Witty Pi 5 HAT+ was added. Your assembly will have three boards: Pi 5 + Witty Pi 5 + G469.)

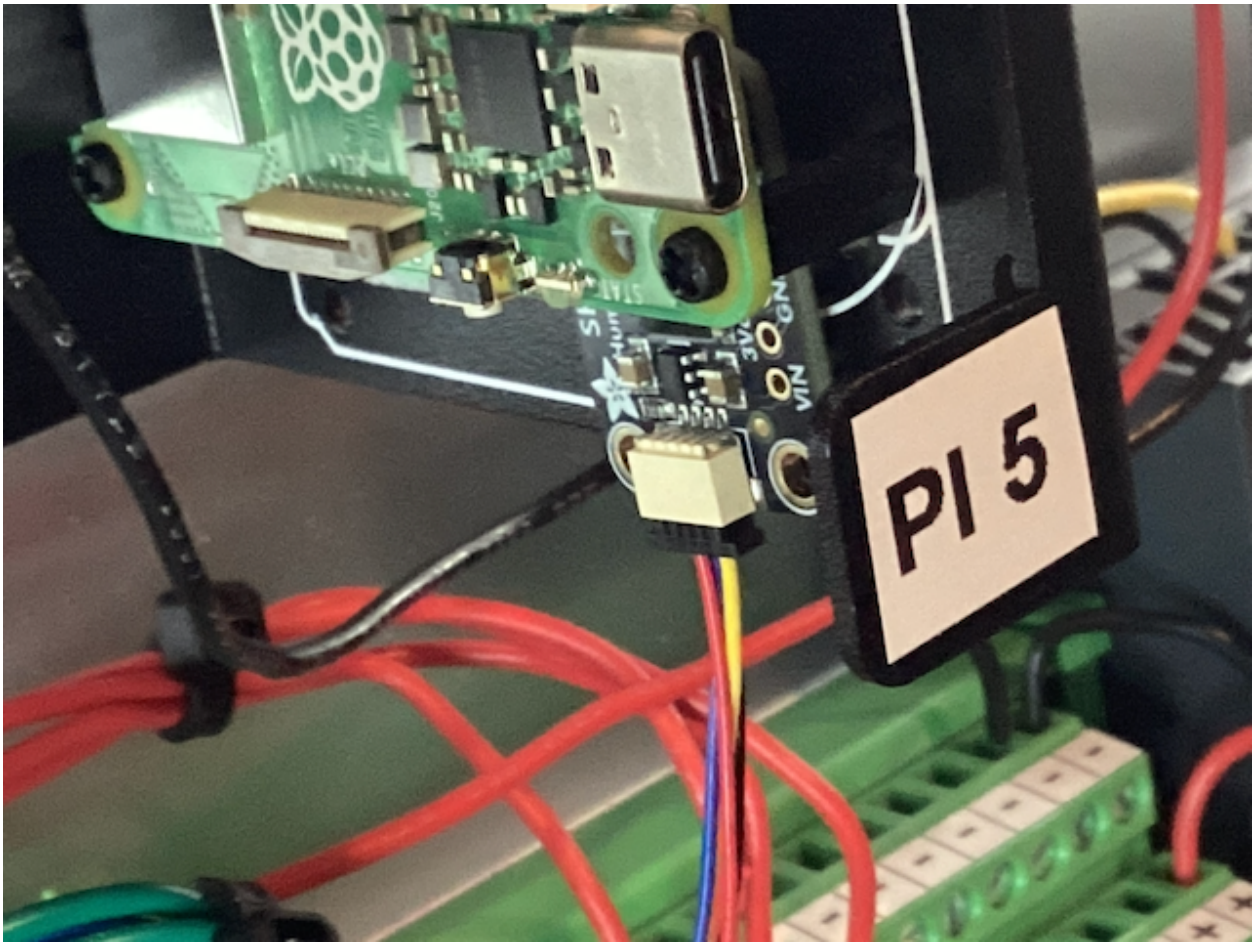

Figure 4: SHT40 temperature/humidity sensor VHB-mounted on Pi DIN rail case with I2C wires

Figure: SHT40 mounted with VHB tape on the Pi case. Position away from heat sources.

4. **Verify all connections before powering on.**

Step 7: Test Mounting Plate Assembly

Power on and verify the complete mounting plate assembly works before proceeding to enclosure work. This is your last chance to fix wiring issues before conformal coating and final assembly.

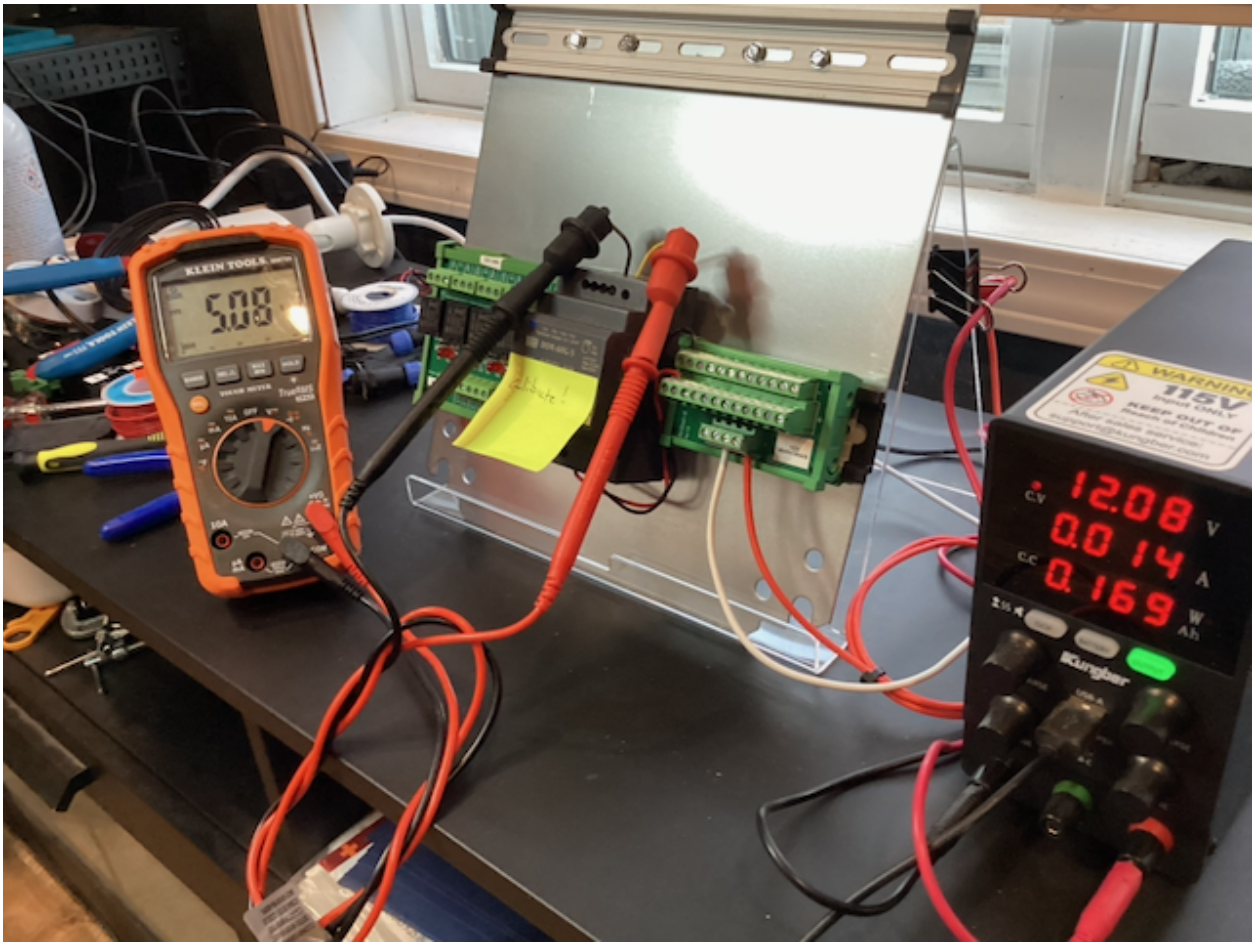

Figure 5: Multimeter reading 5.08V at DDR-60G-5 output with bench supply at 12V

Figure: Verifying 5V output after trim pot adjustment.

- ☐ 12V supply → DDR-60G-5 → Pi boots
- ☐ GPIO 24 → relay CH1 → PoE switch powers on
- ☐ Camera boots and is reachable via Pi
- ☐ Video capture from camera SD to /mnt/usb/incoming works
- ☐ LTE modem detected
- ☐ SHT40 sensor readable via I2C
- ☐ All continuity and isolation checks pass (see GPIO_WIRING.md)

Once the mounting plate assembly is fully tested, proceed to conformal coating (Pre-Assembly Checklist Step 3), then enclosure preparation.

Step 8: Prepare Enclosure and Install Bulkheads

Do not drill the enclosure until the mounting plate assembly is tested.

Tools needed: Drill, sized drill bits or hole saws, step bit (backup), marker

Lesson learned (Sukabumi 2026-04-05): Drill ALL enclosure holes (especially the

antenna hole on top) BEFORE installing the mounting plate. If you drill after the plate is in, you'll need to cup the hole from the inside to catch metal shavings so they don't land on your wired-up components. It works, but it's easier to drill into an empty box. Future builds: do this step with the plate removed.

Drilling tip: Use a sized drill bit or hole saw matched to each hole diameter whenever possible. Step bits are convenient but it's easy to overshoot by one step, and an oversized hole means a sloppy gasket seal. Bulkhead connectors, Gore vents, and antenna mounts all rely on a snug fit to maintain IP rating — slop is the enemy. If you have access to a drill press or a drill guide, use it. A clean, right-sized hole is worth the extra setup time.

1. **Mark and drill enclosure holes:**
   - 2× M12 holes for Gore vents (opposite sides for airflow)
   - 1× 12mm hole for Proxicast puck antenna (enclosure top)
   - 1× hole for CNLINKO ethernet bulkhead (PoE camera)
   - 1× hole for SP13 DC power bulkhead (12V input)
   - 1× 10-12mm hole for status LED light window
   - 1× hole for power button — hole size depends on the button you purchased (12mm, 16mm, 19mm, or 22mm are common)
   - 1× 16mm hole for rain gauge SD16 bulkhead connector
2. **Install bulkheads and glands:**
   - Gore M12 vents (hand-tight plus 1/4 turn)
   - SP13 DC power bulkhead
   - CNLINKO ethernet bulkhead
   - SD16 4-pin bulkhead connector for rain gauge
3. **Install status LED light window (silicone-filled acrylic sandwich):**

   A single WS2812B (NeoPixel) RGB LED provides multi-color status indication using only 1 GPIO data pin and 5V power. The LED is mounted inside the enclosure behind a sealed clear window — no relay channels used.

   Materials:
   - 2× clear acrylic (cast PMMA) squares, ~25mm × 25mm, 2-3mm thick
   - Clear **neutral-cure** silicone sealant (GE Silicone II or equivalent — NOT acetoxy. If it smells like vinegar, don't use it on acrylic.)
   - 1× WS2812B individual pixel PCB
   - IPA (isopropyl alcohol) for cleaning — never use acetone on acrylic

   Procedure:
   1. Drill a 10-12mm hole in the enclosure wall where the LED will be visible
   2. Deburr the hole edges and clean with IPA
   3. Cut two acrylic squares large enough to cover the hole with ~5mm overlap on all sides
   4. Clean acrylic pieces with IPA (not acetone — acetone crazes acrylic)
   5. Apply neutral-cure silicone around the hole perimeter on the **outside** of the enclosure wall
   6. Press the outer acrylic square onto the silicone, centered over the hole
   7. From the inside, squeeze clear neutral-cure silicone into the hole cavity until it's com-

pletely filled — no air bubbles
8. Apply silicone around the hole perimeter on the **inside** of the wall
9. Press the inner acrylic square onto the silicone, centered over the hole
10. Allow to cure 24 hours minimum before water exposure
11. Mount the WS2812B pixel on the inner acrylic face using a small dab of silicone or double-sided tape, LED facing outward through the window

Why solid fill: An air gap between the sheets will fog with condensation in tropical humidity. The solid silicone fill eliminates the air cavity entirely — no condensation surface, no fogging.

Wiring (1 GPIO data pin + 5V power):

- WS2812B VCC → 5V terminal block (after DDR-60G-5 fuse, NOT G469)
- WS2812B GND → G469 GND (any free pin)
- WS2812B DIN (data in) → G469 GPIO 18 (Pin 12)
- All three wires terminate in Dupont female at the LED end (detachable for bottom plate removal)

See **GPIO_WIRING.md Step 4** for full wiring details and verification. See **docs/LED_STATUS_SPEC.md** for color/pattern meanings and config.

4. **Install power button:**

- Install power button in panel, secure with nut
- Connect Dupont female pigtail from J2 to button's male header pins
- Verify strain relief zip tie on pigtail is secure at G469 carrier back
- Label power button clearly as "POWER"

See **GPIO_WIRING.md Step 5** for detailed wiring and build notes.

Step 9: Install Mounting Plate and Connect External Peripherals

1. **Install mounting plate** into enclosure with provided screws
2. **Connect antenna:**
   - Mount Proxicast ANT-122-S02 puck antenna in 12mm hole
   - Route internal SMA cables to modem U.FL connectors
3. **Connect 12V input:**
   - Wire from SP13 DC power bulkhead to TB1 terminal block
4. **Connect rain gauge:**
   - Rain gauge connects through a 4-pin SD16 bulkhead connector (16mm hole)
   - Outside: 18/4 stranded jacketed cable from RG-15 pigtail to bulkhead plug
   - Inside: solid core 22 AWG from bulkhead socket to G469/TB1
   - Pin 1: 12V (TB1), Pin 2: GND (G469 Pin 9), Pin 3: TX→RX (GPIO 14), Pin 4: RX→TX (GPIO 15)
   - See GPIO_WIRING.md Step 6 for full pin map, connector details, and verification checklists
5. **Connect Cat6 outdoor cable:**
   - PoE switch PoE port → CNLINKO bulkhead → Camera

Step 10: Configure Pi Camera Network (15 min)

The Pi serves as DHCP server for the camera network on eth0 using dnsmasq. This avoids the ANNKE's default 192.168.1.x subnet, which conflicts with most WiFi routers.

Note: The SADP utility (Hikvision's camera discovery tool) does not run on ARM Macs — neither natively nor under Parallels. The dnsmasq approach below eliminates the need for SADP entirely.

1. **Set Pi eth0 to a static IP on the camera network:**

   ```
   sudo nmcli con mod "Wired connection 1" ipv4.addresses 192.168.50.1/24
   sudo nmcli con mod "Wired connection 1" ipv4.method manual
   sudo nmcli con up "Wired connection 1"
   ```

   If the connection name differs, find it with `nmcli con show`.

2. **Install and configure dnsmasq as DHCP server on eth0:**

   ```
   sudo apt install dnsmasq
   ```

   Edit `/etc/dnsmasq.conf`:

   ```
   interface=eth0
   bind-dynamic
   dhcp-range=192.168.50.200,192.168.50.254,24h
   dhcp-host=<CAMERA_MAC>,192.168.50.100
   ```

   Important: Use `bind-dynamic` (not `bind-interfaces`). This allows dnsmasq to start before eth0 has carrier — required because the PoE relay may not be on at boot time. Replace `<CAMERA_MAC>` with the camera's MAC address (printed on the camera label, or found via `arp -a` after the camera boots).

   Restart dnsmasq:

   ```
   sudo systemctl restart dnsmasq
   sudo systemctl enable dnsmasq
   ```

3. **Connect and discover the camera:**

   - Connect camera to PoE switch, connect switch uplink port to Pi's Ethernet port
   - Wait 60-90 seconds for camera to boot
   - The camera will receive 192.168.50.100 via DHCP
   - Verify:

     ```
     ping 192.168.50.100
     ```

   - If you need to find the MAC address first, temporarily omit the `dhcp-host` line, restart dnsmasq, let the camera get any IP from the range, then check:

     ```
     cat /var/lib/misc/dnsmasq.leases
     ```

Step 10b: Prepare USB Storage (10 min)

The USB flash drive stores camera uploads and ORC data. It must be formatted as ext4 (for Linux file permissions) and mounted persistently at `/mnt/usb`.

1. **Insert the USB flash drive** into Pi 5 USB-A 3.0 port (blue).

2. **Identify the device:**

   ```
   lsblk
   ```

   The USB drive will appear as `/dev/sda` (or similar). Note the partition name (e.g. `/dev/sda1`).

3. **Unmount if auto-mounted:**

   ```
   sudo umount /dev/sda1   # adjust if different device
   ```

4. **Format as ext4:**

   ```
   sudo mkfs.ext4 -L orc-data /dev/sda1
   ```

5. **Get the UUID** (needed for persistent fstab mount):

   ```
   sudo blkid /dev/sda1
   ```

   Copy the `UUID="..."` value.

6. **Create mount point and add to fstab:**

   ```
   sudo mkdir -p /mnt/usb
   echo 'UUID=<paste-uuid-here>  /mnt/usb  ext4  defaults,noatime,nofail  0  2' | sudo
   ↪  tee -a /etc/fstab
   ```

   The `nofail` option ensures the Pi still boots if the USB drive is missing.

7. **Mount and verify:**

   ```
   sudo mount /mnt/usb
   df -h /mnt/usb
   ```

8. **Create incoming directory for ORC-OS:**

   ```
   sudo mkdir -p /mnt/usb/incoming
   ```

9. **Symlink `/home/pi/Videos` to the USB drive:**

   ORC-OS watches `/home/pi/Videos` for incoming video files. Symlink it to the USB drive so video data stays off the OS SD card:

   ```
   rmdir /home/pi/Videos       # remove empty default directory
   ln -s /mnt/usb/incoming /home/pi/Videos
   ls -la /home/pi/Videos      # verify: Videos -> /mnt/usb/incoming
   ```

   > **Known issue — cross-device link:** ORC-OS uses `os.rename()` to move incoming videos from `~/Videos` to `~/.ORC-OS/tmp/`. If these are on different filesystems (USB vs SD card), the rename fails with `OSError: [Errno 18] Invalid cross-device link` and videos never appear in the web UI. If you hit this, revert to a plain directory on the SD card: `rm ~/Videos && mkdir ~/Videos`. This is an upstream ORC-OS limitation — track in-country to compare with Jakarta's setup.

Step 10c: Configure NTP Server for Camera Time Sync (5 min)

The camera is on an isolated network with no internet — it cannot reach public NTP servers. The Pi must serve NTP so the camera's clock stays in sync.

1. **Install chrony:**

   ```
   sudo apt install chrony -y
   ```

2. **Deploy the camera-net NTP config** from the overlay:

```
sudo mkdir -p /etc/chrony/conf.d
sudo cp pi/shared/etc/chrony/conf.d/camera-net.conf /etc/chrony/conf.d/
```

   This allows NTP clients on 192.168.50.0/24 and enables serving time even when the Pi itself has no upstream NTP (field deployment without internet).

3. **Restart chrony:**

```
sudo systemctl restart chrony
sudo systemctl enable chrony
```

4. **Timezone: leave as UTC (do NOT change).** ORC-OS requires UTC. Changing to a regional timezone (e.g. Asia/Jakarta) breaks the ORC API's timestamp parsing. Verify with:

```
timedatectl | grep "Time zone"   # should show Etc/UTC or GMT
```

~~Step 10d: FTP Server~~ — REMOVED

> **This step is no longer needed.** Video capture uses RTSP pull via the `orc-capture` script, not camera FTP push. The ANNKE C1200 does not support the scheduled event triggers required for FTP-based capture on a 15-minute duty cycle. See ISS-003 in ISSUE_LOG.md for full history.
>
> vsftpd can be disabled: `sudo systemctl disable --now vsftpd`

Step 11: Configure PoE Camera Settings (30 min)

Note: ANNKE C1200 is factory-sealed IP67. No housing assembly required.

The camera is configured entirely via `camtool.py` using ISAPI (Hikvision HTTP REST API). Configuration files are version-controlled in `camera/common/` and pushed to the camera as a batch. No web UI needed.

Pre-requisite: Ensure `camera/.env` exists with `CAMERA_PASSWORD` set (see `camera/.env.example`). FTP password is not needed — capture is via RTSP.

1. **Insert MicroSD card** into the camera's SD slot (required for local recording — the Pi downloads clips from SD via RTSP playback).

2. **Format the MicroSD card** via ISAPI:

```
# Check camera detects the SD card
curl --digest -u admin:<password> http://192.168.50.100/ISAPI/ContentMgmt/Storage

# Format (takes ~60s for 64GB)
curl --digest -u admin:<password> -X PUT \
  http://192.168.50.100/ISAPI/ContentMgmt/Storage/hdd/1/format
```

3. **Change the default admin password** on the camera web interface:
   - From Pi: `http://192.168.50.100`
   - Default credentials: admin / admin
   - Change password immediately — must match `CAMERA_PASSWORD` in `.env`

4. **Push all camera configs via camtool.py:**

```
cd spring_2026_ID/camera
python3 camtool.py push sukabumi-cam1
```

This pushes the following settings (from `camera/common/`):

Setting	Value	Why
Resolution	1920x1080	ORC recommended (RS docs)
Codec	H.264 Main	Broad compatibility
Bitrate	16 Mbps CBR	Camera maximum; RS recommends 20, confirmed 15 acceptable
Frame rate	12.5 fps	Camera max at 1080p
Audio	Disabled	Not needed for ORC
IR cut filter	Auto	IR at night, visible light by day
Supplement light	IR only	No white LEDs (no visible illumination)
Motion detection	Disabled	Not used — continuous recording instead
NTP server	192.168.50.1 (Pi)	Camera has no internet access
Timezone	UTC	Must match Pi — ORC-OS requires UTC
OSD text overlay	Disabled	On-screen text interferes with ORC image analysis
OSD datetime overlay	Disabled	On-screen timestamps interfere with ORC image analysis

Note: FTP config is not pushed. Video capture uses RTSP pull via `orc-capture`, not camera FTP push. See ISS-003.

Warning — settings revert on power cycle: The ANNKE C1200 resets `supplementLightMode` from `irLight` back to `eventIntelligence` and may re-enable OSD overlays after every reboot. The `orc-capture` script enforces both settings on every wake cycle before capturing video:

- `supplementLightMode` → `irLight` (prevents white LED flashes)
- OSD overlays (date/time, channel name) → disabled (text in video interferes with ORC image analysis)

If you test the camera manually (without `orc-capture`), verify with:

```
curl --digest -u admin:<password> http://192.168.50.100/ISAPI/Image/channels/1 | grep supplementL
curl --digest -u admin:<password> http://192.168.50.100/ISAPI/System/Video/inputs/channels/1/ove
```

5. **Set recording schedule to continuous (CMR):**

```
# Get current recording config
curl --digest -u admin:<password> \
  http://192.168.50.100/ISAPI/ContentMgmt/record/tracks/101 > /tmp/track.xml

# Change MOTION to CMR in all ScheduleAction blocks
sed -i 's/ActionRecordingMode>MOTION/ActionRecordingMode>CMR/g' /tmp/track.xml

# Push back
curl --digest -u admin:<password> -X PUT \
  -H "Content-Type: application/xml" -d @/tmp/track.xml \
  http://192.168.50.100/ISAPI/ContentMgmt/record/tracks/101
```

6. **Verify configuration:**

```
python3 camtool.py diff sukabumi-cam1
```

7. **Test video capture pipeline:**

```
# Wait ~30s for recording to accumulate, then download a 5s clip
START=$(date -d '15 seconds ago' +%Y%m%dT%H%M%S)
END=$(date -d '10 seconds ago' +%Y%m%dT%H%M%S)
FNAME="video_$(date -d '15 seconds ago' +%Y%m%d_%H%M%S).mp4"

ffmpeg -y -rtsp_transport tcp \
  -i "rtsp://admin:<password>@192.168.50.100:554/Streaming/tracks/101?starttime=
  ↪ ${START}&endtime=${END}"
  ↪ \
  -t 5 -c:v copy -an /mnt/usb/incoming/$FNAME

# Verify: 1080p, ~16 Mbps, ~5 seconds
ffprobe /mnt/usb/incoming/$FNAME
```

8. **Verify IR function:**
   - Cover camera lens (simulate darkness)
   - IR LEDs should illuminate (visible glow from IR array)
   - Download a clip — verify IR-lit image (grayscale)

Figure 6: ANNKE C1200 PoE camera mounted on test pole with Cat6 cable

Figure: Camera pole mount test setup.

Step 11b: Enable Capture Service (5 min)

The `orc-capture` service runs once per boot cycle: powers on the PoE relay, waits for the camera, captures 5 seconds of video via RTSP, validates the file against a quality gate, delivers it to the ORC-OS incoming directory, then powers off the PoE relay.

> **Important:** The ORC-OS `orc-gpio-relays` service must stay **disabled**. It uses active-low relay logic, which is inverted from our Electronics-Salon relay module (active-high). `orc-capture` controls the relay directly via `poe-relay`.

1. **Deploy the service files** from the overlay:

```
sudo cp pi/shared/usr/local/bin/orc-capture /usr/local/bin/orc-capture
sudo chmod +x /usr/local/bin/orc-capture
sudo cp pi/shared/etc/systemd/system/orc-capture.service /etc/systemd/system/
sudo systemctl daemon-reload
```

2. **Enable orc-capture and disable orc-gpio-relays:**

```
sudo systemctl enable orc-capture
sudo systemctl disable orc-gpio-relays
```

3. **Test the full capture cycle:**

```
poe-relay off                   # start with camera off
orc-capture --dry-run           # relay on → wait → capture → validate → relay off
```

4. **Verify with preflight:**

```
orc-preflight
```

Should show:

- `[PASS] orc-capture service enabled`
- `[PASS] orc-gpio-relays disabled (correct — orc-capture owns relay)`

Step 11c: Enable Sensor Logging Service (5 min)

The `orc-sensors` service reads the SHT40 temperature/humidity sensor (and any future sensors) on a configurable interval and appends readings to daily CSV files. It runs as a timer-triggered oneshot — the timer ticks every 60 seconds and each sensor's own interval is checked before reading.

1. **Deploy the sensor files** from the overlay:

```
sudo mkdir -p /etc/orc-sensors /usr/local/lib/orc-sensors /var/log/orc/sensors
sudo cp pi/shared/etc/orc-sensors/sht40.conf /etc/orc-sensors/
sudo cp pi/shared/usr/local/bin/orc-sensors /usr/local/bin/
sudo chmod +x /usr/local/bin/orc-sensors
sudo cp pi/shared/usr/local/lib/orc-sensors/sensors_logger.py
↪  /usr/local/lib/orc-sensors/
sudo cp pi/shared/etc/systemd/system/orc-sensors.service /etc/systemd/system/
sudo cp pi/shared/etc/systemd/system/orc-sensors.timer /etc/systemd/system/
sudo chown pi:pi /var/log/orc/sensors
sudo systemctl daemon-reload
```

2. **Enable the timer** (not the service — the timer triggers it):

```
sudo systemctl enable orc-sensors.timer
sudo systemctl start orc-sensors.timer
```

3. **Test a manual read:**

```
orc-sensors
cat /var/log/orc/sensors/sht40_$(date +%Y-%m-%d).csv
```

   Should show a CSV with `timestamp,temp_c,humidity_pct` header and one data row.

4. **Verify timer is scheduled:**

```
systemctl list-timers orc-sensors.timer
```

   Adding future sensors: Drop a new `.conf` file into `/etc/orc-sensors/` with its own `SENSOR_TYPE`, `INTERVAL_SEC`, `CSV_HEADER`, and `LOG_DIR`. Add a matching `read_<type>()` driver function in `sensors_logger.py`.

Step 12: Mount External Components (30 min)

Tools needed: Adjustable wrench, stainless U-bolts

1. **Camera mounting:**
   - Select pole/mounting location with clear river view
   - Use stainless U-bolts to secure ANNKE C1200 bracket
   - Aim camera at target water area
   - Secure Cat6 cable along pole with UV-resistant cable ties
   - Connect Cat6 to CNLINKO bulkhead exterior connector
2. **Rain gauge mounting:**
   - Mount Hydreon RG-15 on pole arm, away from obstructions
   - Built-in mounting holes, no separate bracket needed
   - Mount with dome facing up, away from overhanging surfaces that could drip onto it (no leveling needed — optical sensor, not tipping bucket)
   - Route cable to enclosure
3. **Antenna:**
   - Puck antenna already mounted on enclosure top (Step 1)
   - Verify secure and weatherproof seal

Step 13: Final Assembly & Sealing (15 min)

1. **Cable management:**
   - Route all internal cables neatly
   - Use cable ties to bundle
   - Ensure no cables block Gore vents
2. **Seal bulkheads and glands:**
   - Tighten all bulkheads and glands firmly
   - Apply thin silicone bead around each exterior connection
3. **Final checks:**
   - ☐ All connections secure
   - ☐ Gore vents unobstructed
   - ☐ LEDs visible from outside

- ☐ Power button accessible and labeled
- ☐ Power button wired to Pi 5 J2 header (not G469 terminals)
- ☐ No loose items inside

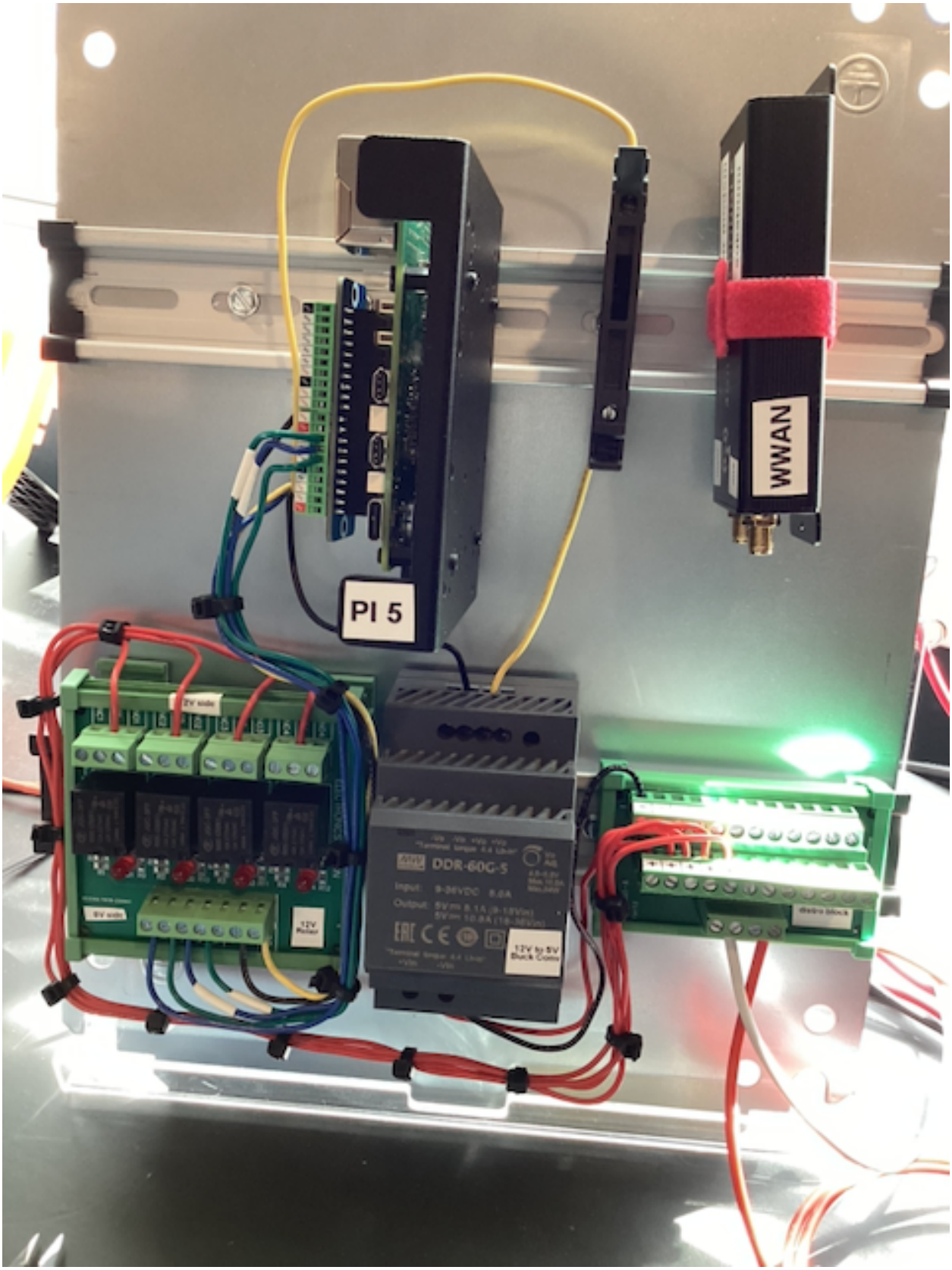

Figure 7: Sukabumi station wiring overview showing signal and power wire routing on DIN rails

Figure: Completed Sukabumi wiring before enclosure installation. (Photo taken before Witty Pi 5 HAT+ was added. Your assembly will have three boards: Pi 5 + Witty Pi 5 + G469.)

4. **Close enclosure:**
   - Verify gasket is clean and seated
   - Close lid, secure all latches/screws

Power-On Procedure

First Boot

1. **Verify solar battery voltage:** Should be >12V
2. **Connect 12V input** to enclosure (SP13 bulkhead)
3. **Press the external power button** (brief press) to power on the Pi 5
4. **Observe:**
   - Pi 5 power LED should light (red)
   - Pi 5 should boot (activity LED flashing green)
   - Status LEDs should indicate boot sequence
5. **Wait 2-3 minutes** for full boot
6. **Check status LEDs:**
   - Green steady = OK
   - Yellow blinking = working (capture/upload)
   - Red steady = camera offline (normal between duty cycles)
   - Red blink = capture failed (check logs)

Run Preflight Checks

After SSH'ing into the Pi, run the preflight check to verify all packages, configs, services, and hardware are correct:

```
orc-preflight
```

This runs `deploy.sh sukabumi --check` (auto-detected from hostname). Review the output — all items should be PASS (WARN is informational). If there are FAILs, run deploy.sh to fix them:

```
cd ~/code/git/openrivercam/spring_2026_ID/pi
bash deploy.sh sukabumi
```

This checks everything first, lists fixes needed, then prompts before applying. Fix any remaining FAILs manually before proceeding.

Verify Camera

1. Enter maintenance mode (long press power button, 3 seconds)
2. Connect to the Pi via Pangolin (`https://arc-00002.openrivercam.com`) or Tailscale
3. SSH into Pi
4. Check camera is reachable: `ping 192.168.50.100`

5. Download a test clip from camera SD:

```
START=$(date -d '15 seconds ago' +%Y%m%dT%H%M%S)
ffmpeg -y -rtsp_transport tcp \
  -i "rtsp://admin:PASSWORD@192.168.50.100:554/Streaming/tracks/101?starttime=
  ↪  ${START}"
  ↪  \
  -t 5 -c:v copy -an /mnt/usb/incoming/test.mp4
```

6. Verify clip: `ffprobe /mnt/usb/incoming/test.mp4` (should show 1080p, ~16 Mbps)

Verify IR Light

1. Cover camera lens with hand or cloth (simulate darkness)
2. Camera IR LEDs should illuminate automatically (visible red glow)
3. Capture test image, verify IR-lit scene
4. Uncover lens, IR should turn off

Verify LTE Connection

1. Check modem detected: `mmcli -L`
2. Check signal: `mmcli -m 0 --signal-get`
3. Test data: `ping -c 3 8.8.8.8`

ORC Software Configuration

All hardware verified, network up, camera reachable. Now configure ORC-OS for automated capture and upstream data sync.

ORC-OS Web UI Initial Setup

Access at `https://arc-00002.openrivercam.com` (via Pangolin).

- ☐ Set ORC-OS web dashboard password on first access
- ☐ **Disk Management** (`/disk_management`):
  - Minimum free space: **5 GB**
  - Cleanup check frequency: **300 seconds** (5 minutes)
- ☐ **LiveORC API** (`/callback_url`) – do this BEFORE daemon settings:
  1. On **LiveORC server** (`https://openrivercam.endlessprojects.info/admin/`): create a site for Sukabumi with GPS coordinates. Note the **Site ID number** (visible in the site detail URL or list). Sukabumi = Site **TBD** (to be created in-country).
  2. On **ORC-OS** (`/callback_url`):
     - Server URL: `https://openrivercam.endlessprojects.info` (no `/admin/` or `/api` suffix)
     - Username and password (LiveORC credentials)
     - Submit – username/password will be replaced by access + refresh tokens
     - Site ID: **TBD** (Sukabumi – deferred to in-country setup)
     - Retry time: set for intermittent LTE connectivity (e.g. 120 seconds)
     - Verify green "callback URL created" banner

- ☐ **Daemon Settings** (`/settings`):
  - Video filename template: `{%Y%m%dT%H%M%S}.mp4` (matches orc-capture output)
  - Parse time from filename: **ON**
  - Verify red confirmation message shows: `/home/pi/Videos/YYYYMMDDTHHMMSS.mp4`
  - Allowed time difference (video water level): **3600 seconds** (NodeORC default)
  - "Shutdown after task": **ON** (Sukabumi is solar/duty-cycled; ORC-OS shuts down after processing)
  - "Reboot after time": **3600 seconds** (1hr safety net – prevents battery drain if processing hangs)
  - Video configuration: select finalized config (after calibration – deferred to field)
  - LiveORC sync: **time series + analysis images** (full video disabled to save bandwidth)
  - Daemon runner: **OFF** until end-to-end test passes, then **ON**
- ☐ **Water Level** (`/water_level`) – deferred to field (requires site survey):
  - Configure retrieval script or manual entry method
  - Set sync tolerance (seconds) between video and water level
- ☐ **Camera config** via camtool.py:
  - Push streaming config (1920x1080, H.264, 16 Mbps CBR, 12.5fps)
  - Push image config (IR-only supplement, auto IR cut filter)
  - Push NTP config (Pi as NTP server: 192.168.50.1)
  - Verify RTSP stream: `orc-capture --skip-relay --dry-run`

Capture Scheduling (ORC-OS Managed)

Video capture is managed by ORC-OS as a systemd TIMER service. The timer fires every 15 minutes and runs `/usr/local/bin/orc-capture` via a thin wrapper script. This gives the ORC team start/stop/enable/disable control from the web UI, while all capture logic and configuration stays in `/etc/orc-capture.conf`.

Architecture: - `orc-capture.timer` – systemd timer, fires every 15 min (`OnCalendar=*:0/15`) - `orc-capture.service` – runs `~/.ORC-OS/services/orc-capture.sh` - `orc-capture.sh` – wrapper that checks `CAPTURE_ENABLED` env var, then `execs` `/usr/local/bin/orc-capture` - `/etc/orc-capture.conf` – runtime config (camera IP, duration, quality gate, etc.) - All service files live in `~/.ORC-OS/services/` and are symlinked to `/etc/systemd/system/`

Service definition is stored in the repo at `pi/shared/orc-capture-service.json` and imported via `orc service import --deploy` during deploy.sh.

Web UI Setup Steps

1. Navigate to **Services** (`/services`) in the ORC-OS web UI
2. Confirm **"capture"** (ORC Video Capture) appears in the service list
   - If missing, import manually via SSH:
     `orc service import --deploy /path/to/orc-capture-service.json`
3. Click on the **capture** service to open the detail page (`/services/4`)
4. Verify the **CAPTURE_ENABLED** parameter is set to **1** (enabled)
5. Click **Enable** first – this creates the systemd timer symlink
6. Click **Start** – this activates the timer
7. Click **Log** to verify captures are running – you should see quality gate PASS messages and "Delivered:" lines every 15 minutes

Start/Stop Behavior ORC-OS manages the capture timer through four controls. The order matters:

Action	What it does
Enable	Creates timer symlink in systemd, timer starts on boot
Start	Activates the timer now, captures fire every 15 min
Stop	Stops the timer, no more captures until Start
Disable	Removes timer symlink from systemd, won't start on boot

To pause captures: Stop, then Disable. **To resume captures:** Enable first (recreates symlink), then Start.

Enable/Disable controls the systemd symlink – Disable physically removes the timer symlink from `/etc/systemd/system/`, and Enable recreates it. Always **Enable before Start** when resuming from a disabled state.

Verification After starting the service, wait for the next :00/:15/:30/:45 mark, then: - Check the **Log** tab on the service detail page for a successful capture - Check the **Videos** page to confirm the new file appears - On SSH: `systemctl list-timers orc-capture.timer` shows NEXT trigger time

- ☐ Import orc-capture service definition into ORC-OS
- ☐ Verify timer is active: `systemctl list-timers orc-capture.timer`
- ☐ Verify capture runs on timer: check logs after next :00/:15/:30/:45
- ☐ Verify start/stop/enable/disable works from ORC-OS web UI

Witty Pi 5 Power Management (Sukabumi)

Power management uses a split responsibility model:

Component	Responsibility
Witty Pi 5	Owns the wake/startup schedule (configured via `wp5` interactive menu)
ORC-OS	Owns shutdown decisions (`shutdown_after_task` ON, `reboot_after` 3600s watchdog)
systemd	`orc-api.service` starts after `wp5d.service` (prevents time-jump reboot loop)

Sukabumi is solar/duty-cycled. The Witty Pi controls when the Pi wakes up, and ORC-OS shuts it down after the capture-process-upload cycle completes. The Witty Pi ON window acts as a backstop – if ORC-OS fails to shut down (e.g. processing hangs), the Witty Pi ON window expires and the Pi powers off anyway, preventing battery drain.

```
# Verify systemd ordering (set by deploy.sh)
grep "wp5d" /etc/systemd/system/orc-api.service
# Expected: After=network.target wp5d.service
```

- ☐ Verify `orc-api.service` has `wp5d.service` dependency

Schedule Files Three `.wpi` schedule files are available, each defining a different duty cycle:

Schedule File	ON Window	OFF Window	Use Case
`prod_15.wpi`	10 min	5 min	**Default production** – captures every 15 min
`prod_30.wpi`	25 min	5 min	**Low-solar fallback** – captures every 30 min, reduces cycles/day
`maint.wpi`	Always on	Never	**Debugging/maintenance** – Pi stays awake indefinitely

How it works: The Witty Pi ON window defines how long the Pi is allowed to stay powered. ORC-OS ("Shutdown after task" = ON) shuts down the Pi as soon as capture + processing + upload finishes – typically well before the ON window expires. The ON window is a safety net, not the normal shutdown mechanism.

- `prod_15.wpi` (default): 10 min ON / 5 min OFF. ORC-OS normally shuts down within 3-5 min. The remaining ON time is buffer for slow uploads or retries. If ORC-OS hangs, the Pi powers off at the 10-min mark.
- `prod_30.wpi`: 25 min ON / 5 min OFF. Use during extended rainy season or when solar budget is tight. Halves the number of wake cycles per day.
- `maint.wpi`: Always-on. Use for field debugging, calibration, or initial setup. Remember to switch back to a production schedule before leaving site.

Loading Schedule Files onto Witty Pi

1. Shut down the Pi (Witty Pi stays powered from external supply)
2. Connect laptop USB cable to Witty Pi's USB-C port – drive appears as "Witty Pi 5"
3. Copy `.wpi` files into `schedule/` on the emulated drive
4. Safely eject, disconnect cable, boot Pi
5. Run `wp5` option 6 to select the active schedule

- **Do NOT connect the Pi's USB-A to the Witty Pi USB-C** – causes power feedback reboot loop

Switching Schedules in the Field To change the active schedule (e.g. switching from `prod_15` to `maint` for debugging):

1. SSH into the Pi (via Pangolin or Tailscale)
2. Run `wp5`
3. Select option **6** (Choose schedule script)
4. Select the desired `.wpi` file
5. Confirm the schedule is loaded (wp5 displays next ON/OFF times)

6. If switching TO a production schedule from `maint.wpi`, verify ORC-OS daemon settings: "Shutdown after task" = **ON**, "Reboot after" = **3600s**
7. If switching TO `maint.wpi` for debugging, consider setting "Shutdown after task" = **OFF** temporarily so the Pi stays awake

- ☐ Load all three `.wpi` schedule files onto Witty Pi
- ☐ Set active schedule to `prod_15.wpi` (default)
- ☐ Verify schedule is active: `wp5` shows correct next ON/OFF times
- ☐ Test duty cycle: observe Pi shutdown after ORC-OS task completes, then wake at next scheduled ON window

Pangolin Remote Access

Pangolin provides remote HTTPS access to the ORC-OS web UI via a tunneled reverse proxy. It is pre-installed on the Rainbow Sensing ORC-OS image. Configuration is handled entirely through the ORC-OS web UI (`/pangolin` page) – no software installation needed. If using a different base image, operators must provision their own remote access service.

Important: the Pangolin server URL is not the same as the end-user proxy URL. The server URL (e.g. `https://pangolin.openrivercam.com`) is the Pangolin management server where Newt authenticates. The proxy URL (e.g. `https://arc-00002.openrivercam.com`) is the public-facing address that end users visit to reach the ORC-OS dashboard. Enter the **server URL** in the ORC-OS configuration, not the proxy URL – using the proxy URL will cause token decode errors (the proxy returns HTML, not the expected JSON).

- ☐ Coordinate with ORC team to create site credentials on the Pangolin dashboard
- ☐ Configure Pangolin connection via ORC-OS web UI (server URL, site ID, secret)
- ☐ Verify tunnel connects (check Pangolin dashboard shows site online)
- ☐ Verify HTTPS proxy URL loads ORC-OS web UI from offsite: `https://arc-00002.openrivercam.com`

Note: Only HTTPS proxy mode is used (for remote dashboard access). arm64 WireGuard tunnel mode has a known issue (fosrl/newt#237) but this does not affect HTTPS proxy mode.

Tailscale will be evaluated in-country as an alternative for SSH access. May be a better fit in some situations, but not usable in countries that disallow third-party VPN services.

LiveORC Server Check

LiveORC server: `https://openrivercam.endlessprojects.info/` Hosted on AWS. Startup script: `/opt/LiveORC/start-liveorc.sh`

- ☐ Verify AWS LiveORC instance is running
- ☐ Confirm API endpoint is reachable (HTTPS)
- ☐ Create Sukabumi site in LiveORC admin (Site ID: TBD – deferred to in-country)
- ☐ Test data upload (manual or via daemon – happens during end-to-end test)

If SSL cert expires (Let's Encrypt, auto-renews inside container): 1. SSH into AWS instance (SSM session or SSH) 2. Stop the stack: `cd /opt/LiveORC && sudo ./liveorc.sh stop` 3. Restart with SSL: `sudo ./start-liveorc.sh` 4. Verify: `curl -I https://openrivercam.endlessprojects.info/` 5. Should return HTTP 302 with valid cert. If cert still expired, check that ports 80 and 443 are open in the AWS security group (Let's Encrypt HTTP-01 challenge needs port 80).

Note: The device won't appear in LiveORC's device list until the daemon sends its first data sync. This is expected – it registers on first upload.

End-to-End Verification

Step 1: Confirm capture produces correctly named files Run a manual capture and verify the filename matches the ORC-OS template:

```
orc-capture
ls -l /home/pi/Videos/
```

Files must be named `YYYYMMDDTHHMMSS.mp4` (e.g. `20260406T172012.mp4`). This matches the ORC-OS daemon template `{%Y%m%dT%H%M%S}.mp4`.

If files are named `video_YYYYMMDD_HHMMSS.mp4` instead: the deployed `/etc/orc-capture.conf` is stale. Re-run `deploy.sh sukabumi` or manually change `FILENAME_PREFIX=video` to `FILENAME_TEMPLATE=%Y%m%dT%H%M%S` in `/etc/orc-capture.conf` and update `/usr/local/bin/orc-capture` to match.

Step 2: Verify ORC-OS daemon is active and detecting files The daemon polls `/home/pi/Videos/` every 5 seconds for new files matching the template. It only runs if **both** conditions are met: 1. Daemon settings are configured in the web UI (`/settings`) 2. Daemon runner is set to **ON**

Important: The daemon scheduler initializes at API startup. If you enable the daemon in the web UI after boot, you must restart the API:

```
sudo systemctl restart orc-api
```

Then check the logs to confirm:

```
journalctl -u orc-api --no-pager -n 20 | grep -i "video_check_job\|daemon"
```

You should see:

```
Daemon settings found: setting up interval job "video_check_job" with
path: /home/pi/Videos and file template: {%Y%m%dT%H%M%S}.mp4
```

Step 3: Confirm videos appear in ORC-OS web UI Run a capture while the daemon is active:

```
orc-capture
```

Within 5 seconds, the video should appear in the ORC-OS web UI video list. Check the API logs for confirmation:

```
journalctl -u orc-api --no-pager -n 10 | grep "Found file"
```

You should see: `Found file: /home/pi/Videos/YYYYMMDDTHHMMSS.mp4 with timestamp ..., adding to database.`

Videos may show error status if no water level data or video config is set yet – this is expected during bench testing.

Step 4: Full pipeline (deferred to field)

- ☐ orc-capture runs successfully (relay -> camera -> RTSP -> quality gate -> file saved)

- ☐ Daemon picks up captured video and processes it
- ☐ Data syncs to LiveORC server
- ☐ LED status shows green (healthy)
- ☐ Leave running for soak test (minimum 4 hours, ideally overnight)

In-Country TODOs

Items to verify or adjust during field deployment:

- ☐ **Cross-device link check:** If `~/Videos` is symlinked to USB, verify videos appear in the ORC-OS web UI Videos tab after capture. If they don't, check `journalctl -u orc-api` for `Errno 18` and revert to SD card: `rm ~/Videos && mkdir ~/Videos`. Compare with Jakarta setup.
- ☐ **OSD overlays:** Verify camera has OSD disabled (date/time and channel name). Run `orc-capture` and confirm "OSD overlays already disabled" in the log output.
- ☐ **Capture settings review:** Review capture quality gate thresholds based on deployment distance and scene. Evaluate whether resolution, framerate, or codec can be traded to raise the bitrate floor to 15 Mbps or better (current floor: 12 Mbps in `/etc/orc-capture.conf` `MIN_BITRATE_KBPS`). Camera max is 16 Mbps CBR.

Sensor Field Testing

Run `deploy.sh sukabumi` to deploy sensor configs + w1-gpio overlay. **Reboot required** after first deploy (1-Wire overlay needs kernel reload).

RG-15 rain gauge (UART): - [ ] Verify UART wiring: `minicom -D /dev/ttyAMA0 -b 9600` – type `R`, expect `Acc ... mm` - [ ] Verify orc-sensors reads it: `sudo orc-sensors` – check journal for `rg15:` line - [ ] Verify CSV: `cat /var/log/orc/sensors/rg15_$(date +%Y-%m-%d).csv` - [ ] Verify state file: `cat /var/lib/orc-sensors/rg15_acc.txt` - [ ] Simulate rain (pour water on dome), verify acc_mm increases on next read - [ ] **Verify always-on power:** put Pi to sleep (`sudo halt`), then measure 12V at TB1 with multimeter – it should still be present. The RG-15 must stay powered during sleep to accumulate rainfall. If it loses power, rainfall between cycles is lost.

DS18B20 temperature probe (1-Wire): - [ ] Verify device detected: `ls /sys/bus/w1/devices/28-*` - [ ] Verify raw reading: `cat /sys/bus/w1/devices/28-*/temperature` - [ ] Verify orc-sensors reads it: `sudo orc-sensors` – check journal for `ds18b20:` line - [ ] Verify CSV: `cat /var/log/orc/sensors/ds18b20_$(date +%Y-%m-%d).csv` - [ ] Sanity check temp against SHT40 reading

SHT40 (verification only): - [ ] Verify: `sudo orc-sensors` – check journal for `sht40:` line - [ ] Verify CSV: `cat /var/log/orc/sensors/sht40_$(date +%Y-%m-%d).csv`

Troubleshooting

See `TROUBLESHOOTING.md` for detailed diagnostics.

Quick Checks:

Symptom	Check
No boot	Battery voltage, fuses, DDR-60G-5 output, 5V/GND wiring to Pi GPIO, power button pressed? J2 pigtail Dupont connectors seated? Strain relief intact?
No camera	PoE switch powered? GPIO 24 driving relay? Camera IP reachable? Ethernet cables?
No IR	Cover lens to trigger. Check camera IR settings in web UI.
No LTE	Antenna tight? SIM inserted? IMEI registered?
No rain data	UART connection? GPIO 14/15 wiring?

Maintenance Notes

Monthly

- Visual inspection of enclosure seals
- Check antenna connection
- Verify status LEDs functioning

Quarterly

- Clean camera lens (through housing if sealed)
- Check bulkhead tightness
- Verify rain gauge not clogged

Annually

- Inspect conformal coating on PCBs
- Check battery health (solar system)
- Replace any degraded cable ties

Site-Specific Configuration

Record after installation:

Parameter	Value
Pi hostname	
Pi IP (if static)	
WiFi hotspot SSID	N/A (hotspot not available)
WiFi hotspot password	N/A (hotspot not available)
LTE SIM number	
Modem IMEI	

Parameter	Value
Camera IP address	192.168.50.100
Video capture method	RTSP pull via orc-capture (not FTP)
Camera NTP source	Pi at 192.168.50.1 (chrony)
Camera stream	1920x1080, H.264, 16 Mbps CBR, 12.5fps
Camera recording	Continuous (CMR) to MicroSD
Camera illumination	IR only (no white LEDs)
GPS coordinates	
Installation date	
Installer name	

Station Commissioning: Camera Survey & Calibration

Once the station is physically installed and capturing video, the camera must be calibrated with Ground Control Points (GCPs) before ORC-OS can process videos into velocity/discharge measurements. This is done through the ORC-OS web UI.

> **Reference documentation:** The authoritative guide for camera calibration and GCP setup is maintained by the ORC team. Refer to the pyorc user guide for detailed instructions on camera configuration, GCP formats, and cross-section setup. The ORC-OS README covers the web UI workflow.

Prerequisites

- ☐ Camera is in its **final position** — do not move after this point
- ☐ Station is capturing video successfully (end-to-end test passed)
- ☐ Access to ORC-OS web UI (local or via Pangolin)
- ☐ RTK GPS or high-accuracy GPS receiver for GCP survey
- ☐ Staff gauge or pressure gauge for water level reference

Step 1: Fix Camera Position

The camera must be firmly mounted and aimed at the river cross-section before calibration. Any movement after GCP survey invalidates the calibration.

- ☐ Camera bracket tightened — no vibration or play
- ☐ Field of view covers the target cross-section
- ☐ Take a reference photo of the camera position and angle
- ☐ Record camera GPS coordinates

Step 2: GCP Field Survey

Select 6+ ground control points visible in the camera frame. Points should be distributed across the field of view and include both banks if possible.

For each GCP, record: - GPS coordinates (easting, northing, elevation — projected CRS preferred) - Description / photo for identification in the video frame

Also record: - Water level at time of survey (`h_ref`) — read from staff gauge or pressure sensor, in the same vertical datum as the GCP elevations

Save as CSV with columns `x`, `y`, `z` (projected coordinates, e.g. UTM zone 48S / EPSG:32748 for Sukabumi).

- ☐ 6+ GCPs surveyed with GPS
- ☐ Water level (`h_ref`) recorded
- ☐ GCP data saved as CSV

Step 3: Camera Configuration (ORC-OS Web UI)

1. **Create Camera Config** — set image dimensions (1920x1080 for ANNKE C1200)
2. **Upload GCPs** — import your CSV via the control points interface
3. **Match pixels to coordinates** — click each GCP location in a sample video frame to assign pixel coordinates
4. **Fit perspective** — run the camera calibration solver; check the RMS reprojection error (lower is better, aim for < 5 pixels)
5. **Set bounding box** — define the area of interest in the frame

- ☐ Camera config created (1920x1080)
- ☐ GCPs uploaded and matched to pixel locations
- ☐ Perspective fit completed (RMS error: _____ pixels)

Step 4: Cross-Section & Recipe

1. **Create cross-section** — define the river cross-section geometry
2. **Create recipe** — set processing parameters (resolution, window size, etc.)
3. **Create video config** — link camera config, recipe, and cross-section

Refer to the pyorc user guide for parameter guidance.

- ☐ Cross-section defined
- ☐ Recipe created
- ☐ Video config created and linked

Step 5: Test Processing

Upload a captured video and verify it processes successfully through the full pipeline (orthorectification, PIV, discharge calculation).

- ☐ Video processed without errors
- ☐ Results visible in ORC-OS web UI
- ☐ Results uploaded to LiveORC (if callback URL configured)

Document Version: 3.4 **Last Updated:** April 7, 2026 **Changes from v3.2:** - Reinstated Witty Pi 5 HAT+ — Pi 5 ML-2020 battery connector (J5) broke on both boards - Stack reverted to 3-board: Pi 5 (bottom) + Witty Pi 5 HAT+ (middle) + G469 (top) with 16mm standoffs - RTC now uses Witty Pi 5 CR2032 coin cell (I2C 0x51) instead of Pi 5 built-in RTC - Removed Pi 5 RTC battery charging config (dtparam=rtc_bbat_vchg) — not needed with Witty Pi - Removed rechargeable ML/LIR battery from parts list; replaced with CR2032 - Added Witty Pi 5 HAT+

PCB to conformal coating list - Updated masking list: Witty Pi CR2032 holder replaces J5 (BAT) connector

Changes from v3.1: - Pushbutton wiring moved to GPIO_WIRING.md (Steps 5-6) with detailed button requirements and equivalent options - RTC battery section expanded: supports ML-2020, ML-2032, LIR2032, LIR2020 with chemistry-specific charge voltages - Relay module pin assignments updated: VCC → Pin 4, GND → Pin 20 (avoids doubling up with buck converter on Pin 2/Pin 6) - Removed Witty Pi reference from rain gauge verification - Hole sizes for pushbuttons no longer hardcoded (depends on button purchased) - Solid core wire specified throughout (no ferrules)

Changes from v3.0: - Temporarily removed Witty Pi 5 HAT+ (reinstated in v3.3 after Pi 5 RTC connector failures) - DDR-60G-5 feeds Pi 5 directly via hardwired 5V/GND GPIO pins - Changed relay from USB-powered coil to GPIO-triggered (GPIO 24→IN1, powered by G469 Pin 2/Pin 6) - Changed DS18B20 from GPIO 24 to GPIO 4 (Pin 7) - Changed LEDs from direct 3.3V GPIO with 330 Ohm resistors to 12V panel-mount LEDs switched through relay channels (GPIO 17→IN2, GPIO 27→IN3, GPIO 22→IN4) - Removed current-limiting resistors from parts list - **v5.0 update:** Replaced 3x relay-driven 12V LEDs with single WS2812B NeoPixel on GPIO 18 (5V from Pi rail). Frees relay channels IN2-IN4.

Changes from v2.0: - Renamed Pi-EzConnect → Geekworm G469 - Replaced M.2 SSD with SanDisk 256GB USB flash drive - Replaced Planet IPOE-260-12V with LINOVISION PoE Switch + Electronics-Salon relay - Added DDR-60G-5 (12V→5V) and DDR-60G-12 (12V→12V regulated) buck converters - Replaced DFRobot SEN0575 I2C rain gauge with Hydreon RG-15 UART (GPIO 14/15) - Added SHT40 temp/humidity sensor (I2C) and DS18B20 temperature probe (1-Wire) - Replaced SMA bulkheads with Proxicast ANT-122-S02 puck antenna (12mm hole) - Replaced cable glands with CNLINKO ethernet bulkhead and SP13 DC power bulkhead - Removed USB-RS485 adapter